

BizTrack

The Complete System Handbook – what was built, how it works, and how to defend it

Everything in the repository, explained from zero: the architecture, the database (paper-aligned, office-by-office), the permit workflow engine, every feature, the web and mobile apps, the tests, and the deployment stack. With diagrams.

Read parts 1–4 to understand the system. Part 5 is the walkthrough you can demo from. Parts 8–9 are your defense prep. Every claim here matches the code as of this build: 43 migrations · 15 passing tests · 6 LGU offices · web + mobile + API running locally.

Contents

1 · The big picture

What BizTrack is

The repository map

Architecture (diagram)

The stack and why

2 · The database

Sources of truth & precedence

ER overview (diagram)

All tables, by module

Seeded reality: offices, permits, roles

3 · The workflow engine

The status machine (diagram)

The services that run it

Numbering & money

4 · Feature catalog

Everything the system does

5 · The lifecycle, end to end

Apply → pay → review → inspect → permit

Demo accounts & demo script

6 · The clients

The web app

The mobile app

The API surface

7 · Quality & operations

Tests, Bruno, backups, audit

Deployment (diagram)

8 · Truth & gaps

Traceability vs the paper

Honest gaps & can't-dos

9 · Defense prep

Likely questions, ready answers

One-page cheat sheet

Part 1 – The big picture

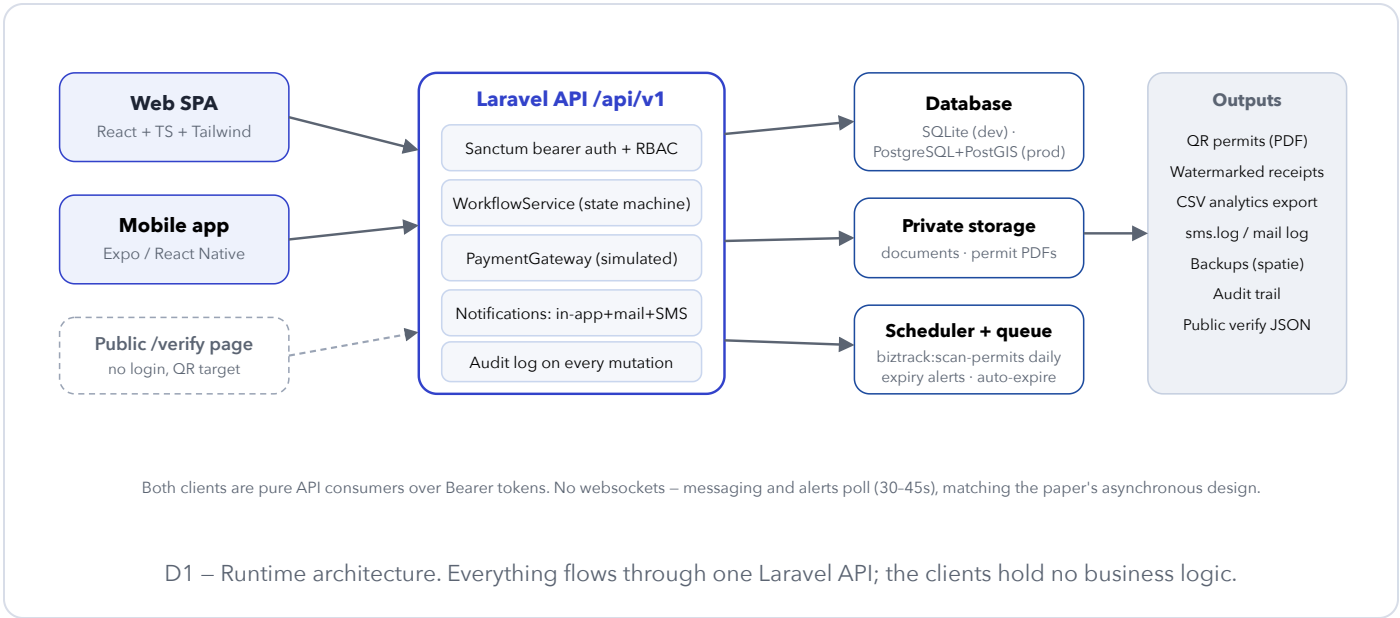
BizTrack is a working, full-stack business-permit system for the City of Malabon: a citizen applies once, six LGU offices process in parallel, and a verifiable permit comes out the other end.

One application covers up to **six permits** (Mayor's/Business, Sanitary, Fire Safety, Occupancy, Environmental, Market Clearance). The system assesses fees, takes a (simulated) online payment, routes the application to every office that owns a requested permit, lets each office review/return/approve, auto-schedules the required inspections, and issues QR-coded permits anyone can verify publicly. Both sides see everything: applicants track status and message officers; officers work department queues; the super admin manages officers, business standing, and analytics.

The repository map

```
biztrack/
├─ api/          Laravel 13 REST API – the entire backend (/api/v1)
│  └─ app/Models, app/Enums      the 30+ domain tables and status enums
│  └─ app/Services                WorkflowService (the state machine),
│  │                             PaymentGateway, NotificationService, Sms/
│  └─ app/Http/Controllers/Api    thin controllers, one per module
│  └─ database/migrations|seeders  the schema + demo data (seed = 1 command)
│  └─ tests/                      Pest suite (15 tests, 63 assertions)
├─ web/          React 18 + TypeScript + Vite + Tailwind – the full UI, rebuilt
│               pixel-faithful to the team's Canva prototype (37 screens/states)
├─ mobile/       Expo + React Native – the owner's mobile app (same API)
├─ r/            the R analytics prototype (paper Features 6 & 7; self-contained)
├─ infra/        Docker: dev (PostGIS+Mailpit) and prod (nginx+fpm+queue+scheduler+db)
├─ docs/         api-contract.md · db-solidification.md · traceability.md ·
│               schema-deltas.md · runbook-deploy.md · bruno/ (71 API requests)
├─ prototype-pages/ all 101 pages of the UI PDF as PNGs (the design source)
└─ dev.sh        one command: runs API + web together
```

Architecture



The stack, and why each piece

Layer	Choice	Why
Backend	Laravel 13 (PHP 8.4), REST, Sanctum bearer tokens	The plan's locked stack; same auth flow for web and mobile, no cookies/CSRF complexity.
AuthZ	Hand-rolled RBAC: <code>roles</code> / <code>permissions</code> / <code>role_permissions</code> / <code>user_roles</code>	Matches the defended schema exactly (no third-party permission package to explain away).
Web	React 18 + TypeScript + Vite + Tailwind v4	SPA served statically; typed against the API contract.
Mobile	Expo + React Native + TypeScript	One JS skillset, on-device via Expo Go, no store accounts needed to demo.
DB	SQLite in dev · PostgreSQL 16 + PostGIS in the prod compose	Zero-setup local dev; the prod target matches the paper. Documented in <code>schema-deltas.md</code> .
Payments	<code>PaymentGateway</code> interface → <code>SimulatedGateway</code>	Real gateways need PCI credentials; the interface swap is designed in, receipts are watermarked "SIMULATED".
Email/SMS	Laravel mail <code>log</code> driver · <code>SmsChannel</code> → <code>LogSmsChannel</code>	Same simulation pattern: the fan-out code is real, the transport is a log file until credentials exist.
PDF/QR	barryvdh/laravel-dompdf + chillerlan/php-qrcode	Permit certificates and receipts server-side; QR embeds the public verify URL.
OCR	smalot/pdfparser (text-layer)	Reads uploaded PDFs and offers prefill <i>suggestions</i> — never auto-commits.
Maps	Leaflet + OpenStreetMap	No API keys, works offline-ish, used for the address pin and the analytics dot-map.
Tests / API docs	Pest · Bruno collection	The plan's quality gates: 15 tests green, 71 documented requests.

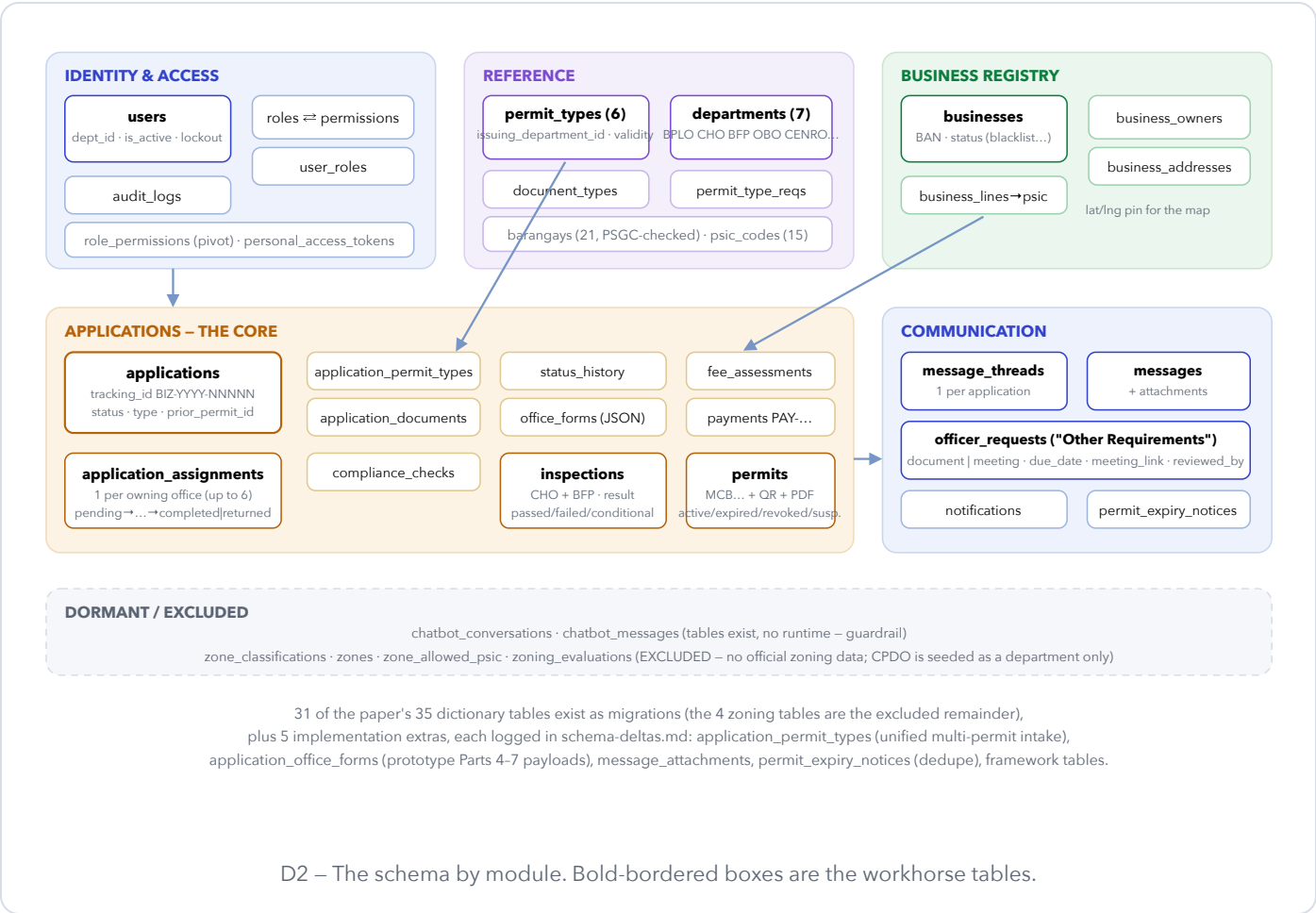
Part 2 – The database

Sources of truth & the precedence rule

- The schema was solidified against **three documents, in this order of authority** (the team's agreed rule):
1. **The UI prototype PDF** ("BizTrack Prototype Linked.pdf", 101 pages) — the most recent team agreement. It's why the DB has **six** permit types/offices.
 2. **The manuscript** (152 pages) — its data dictionary (Tables 27–61, pp. 84–130) defines the 35-table schema, field by field. Every table was diffed against it.
 3. **The sprint plans** (md/00-07) — process and conventions.

The one-line answer to "bakit six permit types, tatlo lang sa paper?" The manuscript names three (business, sanitary, FSIC); the prototype's LGU Section (p.37) — the newer agreement — shows six. The extra three are additive rows in a reference table, breaking nothing. Full citation trail: docs/db-solidification.md .

ER overview



Every module, in one paragraph each

- **Identity & access** — `users` (profile + department + `is_active` + paper's lockout columns), 9 seeded `roles` mapped to the paper's three actors (Business Owner → `business_owner` ; Office Admin → the 7 office roles; Super Admin → `admin`), ~25 dot-namespaced `permissions` (e.g. `application.review` , `fee.adjust` , `owner.manage_status`) joined by two pivots. `audit_logs` records who did what to which record, with before/after snippets, on every mutation.
- **Reference** — 7 `departments` , 6 `permit_types` (each knows its issuing department, fee, validity days, whether it triggers an inspection), 10 `document_types` , and `permit_type_requirements` linking them with the paper's `context` (all/new/renewal/occupancy...) and `is_mandatory` . 21 PSGC-verified `barangays` , 15 illustrative `psic_codes` .
- **Business registry** — `businesses` (registration nos., TIN, `BAN-YYYY-NNNN` , and a `status` of active/flagged/suspended/blacklisted that the super admin controls — blacklisted owners see the red p.6 modal and cannot file), `business_owners` (paper's separate table), one `business_address` with the map pin lat/lng, and `business_lines` → PSIC.
- **Applications core** — `applications` holds the tracking ID, type (new/renewal/amendment), the status machine column, and `prior_permit_id` for prefill. Its satellites: requested permit types (M:N), uploaded documents, an append-only status history (the applicant's timeline), the per-office form payloads (JSON), the fee assessment (line items + officer-adjustable total), and payments.
- **Workflow** — one `application_assignment` per owning office (a 6-permit application spawns 6 queue items), officer `compliance_checks` , auto-scheduled `inspections` for CHO/BFP with a separate result column, and issued `permits` with QR + PDF path + validity computed from `validity_days` .
- **Communication** — a per-application `message_thread` with messages + attachments (polled, no websockets), `officer_requests` in the paper's full shape (document or **meeting** type, due date, meeting link/schedule/duration fields, applicant response, reviewed-by), `notifications` (in-app center; mail + SMS mirrored to logs), and `permit_expiry_notices` so the scheduler never double-sends.

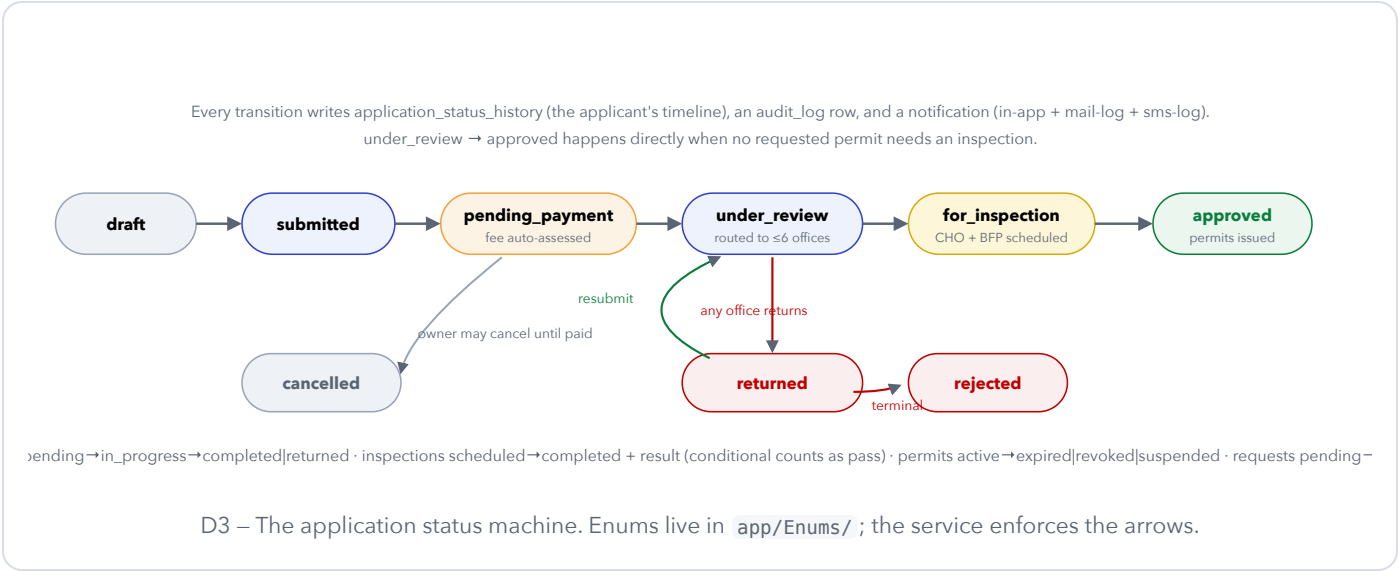
Seeded reality – what a fresh database contains

Office (departments)	Permit type	Prefix	Fee (base + per line)	Inspection?	Demo officer
BPLO — Business Permits & Licensing	Mayor's / Business Permit	MCB	₱1,000 + 150	—	bplo@biztrack.local
CHO — City Health Office	Sanitary Permit / Health Cert.	MCS	₱500 + 50	✓ sanitary	sanitary@biztrack.local
BFP — Bureau of Fire Protection	Fire Safety Inspection Cert.	MCF	₱700 + 80	✓ fire	fire@biztrack.local
OBO — Office of the Building Official	Occupancy Permit	MCO	₱800	—	obo@biztrack.local
CENRO — Environment & Natural Resources	City Environmental Certificate	MCE	₱400	—	cenro@biztrack.local
CMO-MARKET — City Market Administrator	Market Clearance	MCM	₱300	—	market@biztrack.local
CPDO — City Planning (Zoning)	<i>department seeded; zoning permit excluded (no official zone data)</i>				

Plus the demo storyline: Nena's Sari-Sari Store (approved + scannable permit + one permit expiring in 7 days), RxCare Pharmacy (mid-review across 3 offices, with a live message thread and a pending officer request), and demo accounts for every role — all password `biztrack1`, all recreated by one command: `php artisan migrate:fresh --seed`.

Part 3 – The workflow engine

One class — `app/Services/WorkflowService.php` — is the only code allowed to change an application's status. Everything else calls it. That single rule is why the system stays consistent.



The services that run it

Service	Responsibility
<code>WorkflowService</code>	submit (mint tracking ID, assess fee, set RA-11032-style deadline) · onPaymentCompleted (→ under_review + route one assignment per issuing office) · approve/return assignments · reject · resubmit · scheduleInspections (least-loaded inspector, +2 working days) · recordInspection · approveAndIssue (one permit per type, validity from <code>validity_days</code>) · adjustFee · assignOfficer.
<code>PaymentGateway</code>	Simulated charge: instant <code>completed</code> payment with a <code>PAY-YYYY-NNNNNN</code> reference; the interface is the swap-point for a real gateway.
<code>NotificationService</code>	Fan-out for every event (status change, message, request, issuance, fee change, expiry): in-app row + mail (log driver) + SMS (<code>storage/logs/sms.log</code>). Generic payloads, no PII.
<code>biztrack:scan-permits</code> (scheduler, daily)	60/30/7-day expiry reminders (deduped via <code>permit_expiry_notices</code>), auto-flips past-due permits to <code>expired</code> , sends renewal-due notices.
<code>Audit + Numbering</code> (support)	Audit row per mutation · all ID formats: <code>BIZ-2026-00001</code> , <code>MCB-2026-000142</code> , <code>PAY-2026-000004</code> , <code>BAN-2026-0001</code> .

Money

Fee = for each requested permit type: `base_fee + per_line_surcharge × (lines of business)` . Example — Nena requests all six permits with 2 business lines: `1000+300 + 500+100 + 700+160 + 800 + 400 + 300 = ₱4,260` , itemized on the Tax Order of Payment, payable by simulated GCash/Maya/Card, receipt PDF watermarked "SIMULATED PAYMENT". Officers holding `fee.adjust` can re-assess before payment.

Part 4 – Feature catalog

Feature	How it actually works	Where
Unified application	One wizard, up to 6 permits, one BIZ- tracking ID; drafts saved server-side and resumable.	web /apply · ApplicationController
Per-office forms (Parts 4–7)	Sanitary (SP-2026-...), CEC, FSIC, Occupancy forms from prototype pp. 40–43, stored as JSON in application_office_forms .	OfficeFormStep.tsx · office-forms endpoints
Prefill (renewal/amendment)	GET /businesses/{id}/prefill returns the business + last permit + suggested types; wizard pre-populates and links prior_permit_id .	wizard + BusinessController
OCR-lite suggestions	Uploaded PDFs are text-parsed; detected business name / registration no. / validity appear as a dismissible "use it?" banner. Never auto-applied.	OcrLite + upload response
Simulated payment	Method chips (GCash/Maya/Card) → instant completed payment → auto-routing. Receipt PDF watermarked.	/pay · PaymentGateway
Six-office routing	On payment, one assignment per issuing department of each requested type — each office sees only its own queue.	routeToDepartments()
Officer review	The admin-review sheet: real data in read-only form fields, documents, consent bar, tan FOR-OFFICE-USE box (BAN, PSIC chip, assessed fee), Approve / Return-with-remarks / Reject, remark bubbles.	web /queue/:id
Inspections	Auto-scheduled for CHO/BFP on full review approval; inspector conducts pass / conditional / fail (fail = REMARKS FOR REJECTION modal, re-schedulable).	/inspections
Permit issuance + QR	All approvals + inspections passed → one permit per type, numbered, PDF certificate with QR encoding the public verify URL.	approveAndIssue() · /permits/:id
Public verification	/verify/{permit_number} — no login, shows valid/invalid + business + validity. No PII beyond name/barangay.	public route + page
Messaging	Per-application thread (owner ↔ officers), attachments, 30s polling, notifications to the counterparty.	MessagesPanel · messages endpoints
Other Requirements	Officer requests (document or meeting w/ link+schedule fields, due date) → applicant responds (text + file) → officer fulfils/rejects. Letter-style UI per prototype.	/requests
Compliance monitoring	Daily scan: expiry reminders at 60/30/7 days, auto-expiry, renewal-due notices; "Nearing Permit Expiration" in red on the profile.	scheduler command
Business standing	Super admin sets active/flagged/suspended/blacklisted with reason (audited). Blacklisted → red p.6 modal + filing blocked	/admin/owners

	(422).	
Officer management	Officer Assignment table (create, edit, deactivate, activity timeline from audit logs) + per-application OIC assignment.	/admin/users
Analytics	KPIs, monthly line, type bars, Malabon dot-map (green/red business markers), CSV export.	/analytics
Notifications	In-app center + bell; every event mirrored to mail-log and sms.log (simulation pattern).	NotificationService
Audit trail	Every mutation: actor, action, entity, changes, IP Powers the admin audit viewer and officer activity timelines.	audit_logs

Part 5 – The lifecycle, end to end

This exact sequence was executed live against the running system — it's also your demo script.

#	Who	What happens	Behind the scenes
1	Owner	Registers (data-privacy consent) and signs in.	<code>business_owner</code> role; Sanctum token; audit row.
2	Owner	Apply wizard: business info → PSIC lines → address + map pin → picks permits (6 cards) → per-office forms → documents (+ OCR banner) → consent → review → submit.	Draft persists each step; on submit: <code>BIZ-2026-0000N</code> , fee assessed, → <code>pending_payment</code> .
3	Owner	Sees the orange "Payment Status: Pending" card, opens the Tax Order of Payment, pays via GCash.	Simulated gateway: <code>PAY-...</code> completed → <code>under_review</code> → one assignment per office.
4	Each office	Sees the application in its own "Application Verification" queue, opens the admin-review sheet, checks documents, may message the owner or file an "Other Requirements" request, then Approves (or Returns with remarks).	Assignment completed; a Return flips the whole application to returned — owner fixes and resubmits.
5	System	When ALL offices approve: if Sanitary/FSIC were requested → <code>for_inspection</code> with auto-scheduled visits; otherwise → approved immediately.	Least-loaded inspector per office, +2 working days.
6	Inspectors	Conduct: Approve (pass) or the red REMARKS FOR REJECTION modal (fail → reschedule loop).	Conditional counts as pass. All passed → approved.
7	System	Issues one permit per requested type: number, validity from <code>validity_days</code> , PDF certificate with QR.	Owner notified across all channels.
8	Owner / anyone	Owner downloads the permit from the vault; anyone scans the QR → public verify page says Valid permit .	No login required; no PII exposed.
9	System, daily	Expiry scan: reminders at 60/30/7 days, auto-expiry, renewal flags. Renewal prefills from the old permit.	The compliance loop closes back to step 2.

Demo accounts (password `biztrack1`) & run commands

Account	Role / sees
<code>owner@biztrack.local</code> (Nena) · <code>juan@biztrack.local</code>	Business owners — apply, track, pay, message, permits
<code>bplo@</code> · <code>sanitary@</code> · <code>fire@</code> · <code>obo@</code> · <code>cenro@</code> · <code>market@</code>	The six office queues (sanitary/fire also conduct inspections)
<code>admin@biztrack.local</code>	Super admin — analytics, Officer Assignment, Business Owner Status, audit logs
<code>inactive@biztrack.local</code>	Deactivated account (login → 403, for the negative demo)

```
# run everything
./dev.sh                # API :8080 + web :5173, freshly seeded
# prove it
cd api && php artisan test      # 15 passed (63 assertions)
php artisan migrate:fresh --seed  # rebuild the world
php artisan biztrack:scan-permits  # run the compliance scan by hand
# mobile
cd mobile && npm i && npx expo start  # scan QR in Expo Go (set EXPO_PUBLIC_API_URL to LAN IP)
```

Part 6 – The clients

The web app – rebuilt pixel-faithful to the prototype

After a full audit of all 101 prototype pages, the web UI was rebuilt to its visual language: the royal-blue icon rail (#3242ca), periwinkle canvas (#d1dbeb), floating white cards, pill buttons, filled light-blue inputs, serif display accents ("Application Status", ₧ amounts), blue-header modals with split footers, colored status-bar cards (orange/yellow/red/green), and the chatbot bubble. 37 screens/states were captured and verified against their source pages (they're in `bt-figma-shots/` and the board PDFs at repo root).

Area	Routes
Auth	<code>/login</code> (split City-Hall photo) · <code>/register</code> · <code>forgot/reset/verify-email</code>
Owner	<code>/dashboard</code> (4 action tiles) · <code>/apply</code> (dynamic "Part n of N" wizard) · <code>/applications(+/:id,/pay)</code> (Permit Tracking accordions, status cards, messages) · <code>/drafts</code> · <code>/payments</code> · <code>/permits(+/:id)</code> (Profile + certificate w/ QR) · <code>/requests</code> · <code>/notifications</code> · <code>/settings</code>
Officer	<code>/queue(+/:id)</code> (Application Verification + admin-review sheet) · <code>/inspections(+/:id)</code>
Admin	<code>/analytics</code> · <code>/admin/users</code> · <code>/admin/owners</code> · <code>/admin/audit-logs</code>
Public	<code>/verify/:permit_number</code>

The mobile app

Expo + React Native, owner-focused, same API: Login → tabs **Home** (action tiles), **Track** (status cards, **pay from the phone**, resubmit returned filings), **Permits** (validity + verify link), **Alerts** (unread badge, 45s polling) + Requests. Token in SecureStore; 401 auto-logout. Filing stays on web (documented); the app tracks, pays, responds. Verified: `tsc` clean + Expo web bundle export.

The API surface (what Bruno documents – 71 requests)

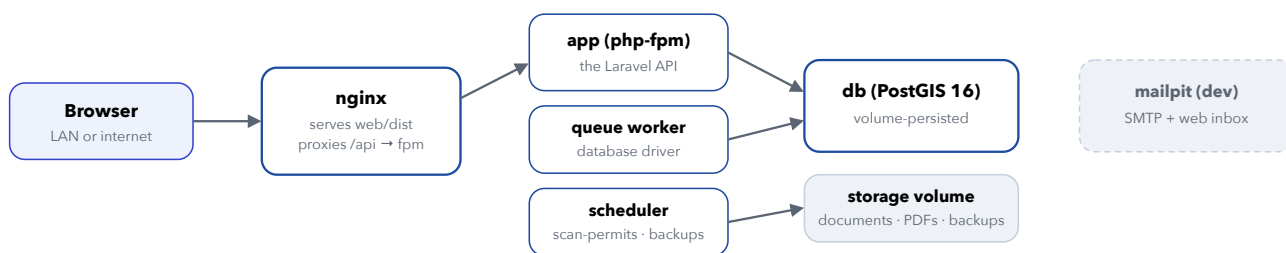
Module	Endpoints (all under /api/v1)
Auth	register · login (throttled) · logout · me · forgot/reset password · email verify/resend
Reference	barangays · psic-codes · departments · permit-types (with requirements) · document-types
Businesses	CRUD (owner-scoped) · prefill
Applications	list/create/show/update · submit · resubmit · cancel · reject · timeline · documents (+OCR) · office-forms · fee · fee/adjust · pay · messages · requests
Officer	assignments (queue/show/approve/return/checks/ assign) · inspections (list/show/conduct/reschedule)
Permits	list/show · pdf · public verify/{number}
Payments	history · receipt
Requests	list · respond · close
Notifications	list · read · read-all
Analytics	summary · export (CSV)
Admin	users CRUD + toggle-active · businesses + status · audit-logs

Bold = added in the gap-closure wave. Full request/response shapes: docs/api-contract.md .

Part 7 – Quality & operations

- **Pest tests** — 15 passing, 63 assertions: auth (good/bad/deactivated), the complete lifecycle including the six-office fan-out, authorization negatives (cross-owner 403, cross-department 403), and the v2 endpoints (messages, requests incl. meeting type, fee adjust, blacklist blocking, prefill). Run: `php artisan test`.
- **Bruno collection** — `docs/bruno/`, 71 requests in 15 folders, with an environment that auto-captures the token on login. Import into the Bruno app and the whole API is clickable.
- **Backups (R32)** — spatie/laravel-backup: nightly schedule + verified manual run (~1.7 MB archive: DB + private documents). Restore drill documented in the runbook and `db-solidification.md`.
- **Audit** — every mutation logged; the admin can browse it, and officer "Details" timelines are built from it.

Deployment



D4 – `infra/docker-compose.prod.yml` : one command on the MISC box or the defense laptop; runs offline after build. Full steps + restore drill: `docs/runbook-deploy.md`.

Part 8 – Truth & gaps

Traceability vs the 152-page manuscript (full detail: docs/traceability.md)

- **Functional R1–R21:** built or partial across the board after the gap-closure wave; only the AI chatbot (R16) is guardrail-deferred (tables exist, no runtime) and zoning sub-scopes are excluded.
- **UCRs 01–20:** all covered except UCR-07 (chatbot, deferred).
- **Data dictionary:** 31 / 35 tables exist; the 4 zoning tables are the excluded remainder; 5 implementation extras logged in schema-deltas.md .

Honest gaps – and what they need

Gap	Status	What unlocks it
Zoning validation (polygons, point-in-polygon)	excluded	Official Malabon zone data (team decision to exclude until then; CPDO seeded, swap path documented).
AI chatbot runtime	dormant	Deliberate guardrail; UI bubble + tables exist, design documented.
Real payment gateway / SMS / SMTP	simulated	PCI credentials, Semaphore key, LGU relay — drivers swap in place, schema unchanged.
Google Meet/Calendar on meeting requests	manual link	Google Cloud OAuth; DB fields already per the paper, links entered manually today.
App-store / TestFlight builds	local only	Apple/Google accounts + EAS; the Expo app runs on-device via Expo Go now.
Figma native board (37 screens staged)	quota-blocked	Figma Starter = 6 MCP calls/month; upgrade or drag bt-figma-shots/ in manually.
Official PSA PSIC 2019 codes	illustrative	The licensed PSA listing; one seeder edit.
Postgres in local dev	by design	Dev = SQLite for zero-setup; prod compose ships PostGIS. Documented delta.

Part 9 – Defense prep

"Walk us through what happens when someone applies."

Use Part 5's table — it is literally the demo script, and every step is reproducible from a fresh seed in under five minutes.

"Why six permit types when the manuscript says three?"

Precedence: the prototype (p.37 LGU Section) is the team's most recent agreement and shows six offices; the manuscript's three are a subset. The extra three are additive reference rows — nothing in the schema changed shape. Citations in `db-solidification.md`.

"Is the database really the paper's database?"

Yes — audited table-by-table against dictionary Tables 27–61: 31 of 35 tables implemented (4 zoning tables excluded for lack of official data), field-level alignments applied (lockout columns, `display_name`, `validity_days`, requirement context, `business_owners`, the full `officer_requests` shape), and every remaining divergence is listed with a reason in `schema-deltas.md`. Nothing is silently different.

"How do you prevent an office from touching another office's work?"

Three layers: route-level permission middleware, department scoping on queue queries, and per-record authorization (we can show the 403 live — it's also a Pest test).

"What about payments — is that real money?"

No — and it says so: the gateway is an interface with a simulated driver, receipts are watermarked "SIMULATED PAYMENT". A real PCI-DSS gateway drops into the same interface with zero schema change. Same pattern for SMS and email (log drivers).

"How do you know the workflow is correct?"

One service owns all transitions; 15 Pest tests cover the happy path (including the six-office fan-out and inspection gating) and the failure paths; every transition is journaled in status history and the audit log.

"Where's the R integration from the other paper?"

The `r/` prototype: synthetic data generator + SPC control charts that catch an injected slowdown + a simmer discrete-event staffing simulation, with its own Shiny dashboard and Plumber API. Self-contained, reproducible from seed 1103 (its own handbook: `BizTrack-R-Guide.pdf`).

One-page cheat sheet

Ask	Answer
Run everything	<code>./dev.sh</code> → web :5173, API :8080, seeded
Prove correctness	<code>php artisan test</code> → 15 passed / 63 assertions
Rebuild the world	<code>php artisan migrate:fresh --seed</code> (43 migrations)
Any login	*@biztrack.local / biztrack1 (owner, juan, bplo, sanitary, fire, obo, cenro, market, admin)
Offices / permits	7 departments · 6 permit types (MCB MCS MCF MCO MCE MCM) · zoning excluded
Status flow	draft → submitted → pending_payment → under_review → (for_inspection) → approved; returned↔resubmit; rejected/cancelled terminal
IDs	BIZ-YYYY-NNNNN · {MCB...}-YYYY-NNNNNN · PAY-YYYY-NNNNNN · BAN-YYYY-NNNN
The one class to read	<code>api/app/Services/WorkflowService.php</code>
The DB review doc	<code>docs/db-solidification.md</code> (Friday artifact)
Requirement mapping	<code>docs/traceability.md</code> · deltas: <code>docs/schema-deltas.md</code>
Deploy	<code>docker compose -f infra/docker-compose.prod.yml up -d --build + runbook</code>

BizTrack System Handbook — generated from the codebase at build time (43 migrations, 15 tests green, web build clean, stack verified in-browser). Companion documents: BizTrack-R-Guide.pdf (the R prototype), BizTrack-Screen-Flow.pdf and BizTrack-All-Screens-v2.pdf (the UI boards), and everything under docs/.